

Counting Sort e Radix Sort

Counting Sort

O **Counting Sort** ordena inteiros contando quantas vezes cada valor aparece. Em vez de comparar elementos, ele usa um vetor auxiliar de contagem.

Funcionamento

Considerando o vetor `[4, 2, 2, 8, 3, 3, 1]`:

1. Encontrar o maior valor do vetor.
2. Criar um vetor `contagem` com posições de `0` até o maior valor.
3. Para cada elemento, incrementar `contagem[elemento]`.
4. Percorrer o vetor de contagem e reconstruir o vetor original na ordem correta.

```
Vetor original
  ↓
Contar ocorrências de cada valor
  ↓
Percorrer as contagens em ordem crescente
  ↓
Vetor ordenado
```

O Count Sort é mais eficiente quando o intervalo de valores é pequeno em relação à quantidade de elementos. A implementação abaixo considera apenas **inteiros não negativos**.

Complexidade do Count Sort

Se `n` é a quantidade de elementos e `k` é o maior valor do vetor, o custo é $O(n + k)$.

Característica	Count Sort
Método	Não comparação
Melhor	$O(n + k)$
Médio	$O(n + k)$
Pior	$O(n + k)$
Memória	$O(k)$
Estável?	Não, nesta versão
In-place?	Não

Radix Sort LSD (Least Significant Digit)

O **Radix Sort LSD** (*Least Significant Digit*) ordena inteiros analisando seus dígitos do menos significativo para o mais significativo. Portanto, começa pela casa das unidades, depois passa para as dezenas, centenas e assim por diante. Em cada posição decimal, ele utiliza uma ordenação estável por contagem.

Funcionamento

No Radix Sort LSD, o algoritmo:

1. Identifica o maior número para descobrir quantas casas decimais serão processadas.
2. Ordena os elementos pelo dígito das unidades usando uma ordenação estável.
3. Repete o processo para as dezenas, centenas e demais casas decimais.
4. Mantém a ordem relativa dos elementos que possuem o mesmo dígito.
5. Para quando todas as casas do maior número forem processadas.

Exemplo para o vetor [170, 45, 75, 90, 802, 24, 2, 66]:

```
Unidades → 170, 90, 802, 2, 24, 45, 75, 66
Dezenas   → 802, 2, 24, 45, 66, 170, 75, 90
Centenas  → 2, 24, 45, 66, 75, 90, 170, 802
```

Essa é a estratégia **LSD**, pois a ordenação começa pelo dígito menos significativo. O Radix Sort LSD abaixo usa base decimal e considera **inteiros não negativos**. A estabilidade da ordenação por cada dígito é necessária para preservar o resultado das casas já processadas.

Complexidade do Radix Sort LSD

Se n é a quantidade de elementos, d é a quantidade de dígitos do maior valor e b é a base utilizada, a complexidade é $O(d(n + b))$. Com base decimal fixa, $b = 10$, então pode ser escrita como $O(dn)$.

Característica	Radix Sort LSD
Método	Não comparação
Melhor	$O(d(n + b))$
Médio	$O(d(n + b))$
Pior	$O(d(n + b))$
Memória	$O(n + b)$
Estável?	Sim
In-place?	Não